

JARVIS — BUILT BY DILIP

FINAL CODEX MASTER IMPLEMENTATION PROMPT — ADDITIONAL RELEASE GATES

JARVIS — BUILT BY DILIP

FINAL CODEX MASTER IMPLEMENTATION PROMPT

99. FINAL RELEASE / PRODUCTION READINESS GATE

Before declaring the project complete, perform a final end-to-end production-readiness review.

Verify:

- The application starts from a clean environment using the documented setup.
- The UI connects to the backend correctly.
- Ollama/model connectivity works when configured.
- The model router selects the intended model.
- Deterministic tools execute before unnecessary LLM calls.
- Memory, cache, RAG, MCP, tools, terminal, Python execution, browser, scheduler, voice, and computer-control subsystems report truthful health states.
- Permission checks are enforced before sensitive actions.
- User scripts execute as separate processes.
- Generated code is not silently executed.
- Important actions are observable and verifiable.
- Errors are surfaced instead of hidden.
- Background processes are cleaned up correctly.
- No secrets are committed.
- No temporary files, model files, virtual environments, or caches are committed.
- README instructions match the actual project.
- Dependency versions and licenses are documented.
- Windows/Linux limitations are documented accurately.
- Tests pass or failures are explicitly documented.
- The Git working tree is clean before release.

Run a final smoke test covering this real workflow:

START JARVIS

→ HEALTH CHECK

→ USER INPUT

→ FAST INTENT

→ MEMORY/CACHE CHECK

→ TOOL OR MODEL ROUTING

→ **ACTION**

→ **OBSERVATION**

→ **VERIFICATION**

→ **RESPONSE**

→ **CLEANUP**

If any critical component fails, fix it before release where technically possible.

Do not call the project production-ready if critical security, startup, routing, or data-integrity problems remain.

100. FINAL DELIVERY, MAINTAINABILITY AND HANDOVER

The finished repository must be understandable and maintainable by another developer.

Create a final handover package containing:

- README.md
- ARCHITECTURE.md
- DEVELOPMENT.md
- SECURITY.md
- DEPENDENCIES.md
- PERFORMANCE.md
- MCP.md
- RAG.md
- COMPUTER_CONTROL.md
- PYTHON_EXECUTION.md
- WINDOWS.md
- LINUX.md
- VOICE.md
- BROWSER.md
- troubleshooting documentation

Document:

- how JARVIS starts
- how models are configured
- how tools are registered
- how permissions work
- how memory works
- how RAG works
- how MCP servers are added/removed
- how Python scripts are discovered and executed
- how computer control works
- how browser automation works
- how to run tests
- how to run diagnostics
- how to add a new tool

- how to add a new platform adapter
- how to add a new model
- how to add a new MCP integration
- how to extend JARVIS for future ESP32/IoT/robotics hardware

Create an explicit:

JARVIS DOCTOR

diagnostic path and a clear troubleshooting section.

Create an explicit:

JARVIS STATUS

view that reports real subsystem health.

Before final delivery:

1. Run tests.
2. Run diagnostics.
3. Run a smoke test.
4. Review security.
5. Review dependency licenses.
6. Review Git status.
7. Commit the final changes.
8. Push to the NEW GitHub repository.
9. Verify the remote repository contains the final commit.
10. Provide the exact run commands and known limitations.

Final output must distinguish:

WORKING

PARTIALLY WORKING

BLOCKED

NOT IMPLEMENTED

NOT TESTED

Never claim functionality that was not actually tested or verified.

The final goal is a maintainable, secure, low-resource, local-first JARVIS that can evolve from computer software into future hardware/IoT/robotics integrations without requiring a rewrite of the core architecture.